CS 417/545 Project 2: Primary-Backup Key-Value Store

Due: 11/06/2026, 11:59 PM (100 points)

Please read the description and instructions carefully.

Goals and Aims

The purpose of this project is to teach you, through practice, how a distributed service survives the loss of one of its servers. In Project 1 you built the client side of a distributed training system: workers that could keep going through RPC timeouts, lost replies, and crashed peers. Those workers had nowhere to safely store the model parameters they were updating. Project 2 fills that gap. You will build the shared, replicated parameter server that the P1 workers were missing.

Concretely, by the end of this project you will have written a key-value store that uses primary-backup replication to keep every write safe even when one server dies in the middle of training. Along the way you will deepen your understanding of state-machine replication: what it means to commit a write to two machines atomically, why the primary must wait for the backup's acknowledgement before replying to the client, and how at-most-once delivery prevents a retried gradient update from being applied twice. You will also measure how fast your parameter server can serve a real stochastic-gradient-descent (SGD) training loop on iLab.

Performance evaluation starts with P2. The perf test runs an SGD training loop against your parameter server and reports steps per second; see Section 2. Start early. See Section 10 for hints and Section 12 for the plagiarism and AI-use policy.

If primary-backup replication is a new concept to you, read `p2-background.pdf` first. It explains state-machine replication, view servers, and split-brain in about 15 minutes of plain-English reading.

1. Building on Project 1

Project 1 made the CLIENT side of a distributed training system robust. Your workers kept the training loop running through RPC timeouts, lost replies, and worker crashes. The missing piece was WHERE the workers read and wrote the model parameters. In P1 each worker held its own copy. A real distributed training system needs one shared, fault-tolerant place where workers push gradient updates and read the latest weights.

Project 2 is that place. Your P2 primary-backup key-value store is a parameter server. Keys are parameter names (`"w0"`, `"w1"`, ..., `"bias"`). Values are the current value of each parameter. `Put` writes an updated weight. `Get` reads the latest weight. `Append` is useful for simple gradient accumulation.

One SGD step from a worker's point of view:

1 `Get` each weight from the parameter server.

2 Compute the local gradient on a mini-batch.

3 `Put` each updated weight back to the parameter server.

If the parameter server crashes mid-training, the backup takes over and the training loop resumes with the latest weights intact. No step is lost. No step is duplicated. This is exactly what Mu Li's "Parameter Server" (OSDI 2014), the early TensorFlow ps/worker mode, and PyTorch's `torch.distributed.rpc` all do under the hood.

What you reuse from P1:

- The retry loop that keeps trying when an RPC times out.
- The xid logic that tags each operation with a unique ID.
- The duplicate-detection table on the server that remembers "I already handled this xid, here is the reply."
- The pattern of a client-side helper type (your Clerk / Worker) that hides retries from the application.

What P2 adds on top of P1:

- Server-side fault tolerance via primary-backup replication (Part 2).
- A view server that decides who is primary when nodes fail (Part 2).
- Client re-routing when the primary changes (Part 2). The clerk now has to re-query the view server, not just retry the same server.

Why your P1 code must be correct for P2 to pass:

If your at-most-once logic from P1 is broken, duplicate gradient updates corrupt the model. Our P2 tests retry the same operation on purpose. If `Put("w0", "0.42")` applies twice because an xid check misses, that weight ends up with the wrong value and the model drifts away from the reference. If the clerk gives up too early on retries, a transient primary failure looks like a dropped update.

If you had bugs in P1 that you worked around before, fix them now. The first week of P2 is a "P1 regression" window: make the P1 Section 3 tests pass cleanly before you touch Part 2 of P2. Office hours in that first week will prioritize P1 debugging.

2. Performance Evaluation

Starting with P2, every project is measured on performance as well as correctness. Correctness is still the floor. Performance is **15 of 100 points.**

For P2 we measure **one number: SGD steps per second** against your parameter server. The workload is a 10-dimensional linear-regression training loop. Each step the client does one `Get("model")`, computes a gradient locally, and does one `Put("model", ...)`. More steps per

second means a faster parameter server, which in a real system means a model that trains in minutes instead of hours.

The perf test also acts as a smoke check: if training does not converge (final loss > 50% of initial loss), the test fails loudly. That almost always means your primary-backup pair silently dropped some updates.

Where we measure: on iLab, so numbers are comparable across the class.

```
ssh netid@ilab1.cs.rutgers.edu
cd ~/cs417/p2
go test -run Perf . 2>&1 | tee perf.out
```

The last line of output is:

```
PERF  sgd_steps_per_sec=<n>
```

Put that number in `Report.md`.

How we grade the 15 points:

- **5 pts** -- `Report.md` contains the SGD-steps-per-second number from an iLab run.
- **10 pts** -- class ranking. If the top submission does X steps/sec, you get full credit at X/2 or better, zero at X/8 or worse, linearly interpolated in between.

If your correctness tests do not pass, performance points are not awarded. We cannot measure something that does not work.

Attend recitation and office hours for optimization hints.

3. Code Structure

You are given four files:

- `sealed.go` -- the RPC wire types (`GetArgs`, `GetReply`, `PutAppendArgs`, `PutAppendReply`, `ForwardArgs`, `ForwardReply`) and the shared constants. Do not modify this file. We diff your `sealed.go` against the reference before anything else runs.
- `p2.go` -- the file you edit. It contains stubs for the `KVServer` and the `Clerk`. Every function body starts as a `panic("not implemented yet")` with an `ADD YOUR CODE HERE` marker inside. Sections are labelled `Part 1` and `Part 2`, matching this writeup. Add any helper types or fields you need.
- `perf_test.go` -- the performance scenario we run on iLab. Do not modify. See Section 2.
- `go.mod` -- module definition. Standard library only. No third-party packages.

Do NOT change the signatures or fields of any type in `sealed.go` or any of the exported functions in `p2.go`. Everything else is yours: add helper functions, helper types, package-level variables, whatever the design calls for. Only the observable behaviour is graded.

Sanity check the day you receive the skeleton:

```
go build .
go test -run TestExample .
```

If both work, your Go environment is fine and you can start Part 1.

Part 1. Single-Server Key-Value Service (25 points)

Part 1 is a Go warm-up and gets you comfortable with the RPC API you will replicate in Part 2.

Implement the following two operations as RPC methods on a `KVServer` type:

```
func (kv *KVServer) Get(args GetArgs, reply *GetReply) error
func (kv *KVServer) Put(args PutAppendArgs, reply *PutAppendReply) error
```

`Get(key)` returns the current value. `Put(key, value)` overwrites. Missing keys return the empty string.

Also implement the client-side `Clerk`:

```
func (ck *Clerk) Get(key string) string
func (ck *Clerk) Put(key, value string)
```

The clerk talks to the server over RPC and retries on network errors.

Note 1: Operations must be at-most-once from the client's perspective. Every call carries an xid; a retried call with the same xid returns the same reply. Reuse the pattern from P1's Section 3.3.

Note 2: The tests will run two clerks concurrently against one server. Keep server state safe under concurrent access.

Part 2. Primary-Backup Replication (60 points)

Add a second server (the backup) and replicate every write to it before responding to the client.

```
func (kv *KVServer) ForwardOp(args ForwardArgs, reply *ForwardReply) error
```

On every client write, the primary:

1. Validates the request (at-most-once check).
2. Applies the write to its own state.

3 Sends a `ForwardOp` RPC to the backup.

4 Waits for the backup's acknowledgement.

5 Replies to the client.

Reads can be served from the primary alone. (A production system can sometimes read from the backup for scale, but keep reads to the primary for grading.)

Both servers start together at the beginning of each test. Mid-run backup joins and view changes are NOT in scope for P2; the next project introduces view servers and failover. See `p3-future-scope.md`.

What the tests check:

- Writes replicate correctly in the happy path.
- Reads reflect every acked write.
- The backup's state matches the primary's after a burst of writes.
- `go test -race` finds no data races.

Note 1: You MUST wait for the backup's ack before replying to the client. Otherwise a primary crash right after the reply would lose the write (even though we do not test crash recovery in P2, the protocol invariant still applies).

Note 2: The exact RPC-field names in `sealed.go` are what the test code builds. Do not add or rename fields.

4. What We Grade

Your code must handle the following cases.

Part 1.

- Two `Get` calls after a `Put(k, v)` return `v`.
- Retrying a `Put` after a lost reply does not apply the operation twice.
- Concurrent clerks do not corrupt the store.

Part 2.

- A write is reflected in the backup's state before the client sees the reply.
- The backup's full key-value state matches the primary's after a burst of writes.
- `go test -race` finds no data races.

5. Running the Service by Hand

`p2.go` has a `main()` that dispatches on `-mode`. Once each part compiles you can run:

```
go run . -mode kvserver -addr localhost:9002
go run . -mode kvserver -addr localhost:9001 -forward localhost:9002
go run . -mode clerk    -primary localhost:9001 -op put -key x -val 7
go run . -mode clerk    -primary localhost:9001 -op get -key x
```

The first process is the backup (no `-forward`, so it just serves local Puts and ForwardOps). The second is the primary (its `-forward` points at the backup). The clerk talks directly to the primary.

6. Testing Your Own Work

Write your own tests in `yours_test.go` as you build each part. Run everything with the race detector:

```
go test -race .
```

7. Grading Policy

Points:

- CS 417 (undergraduates): 25 (Part 1) + 60 (Part 2) + 15 (Performance, Section 2) = 100.
- CS 545 (graduate students): the same 100, plus 15 more for Section 8. Grade is scaled back to 100.

Partial credit is available. If your code compiles, shows serious effort, and works for most of the cases in Section 4, you will earn partial credit even if some tests fail. A failing test costs half of that test's points. The daily grading email is the final word.

8. Additional Work (Required for CS 545)

This section is required for CS 545 (graduate students) only. CS 417 (undergraduates) may implement any part of it for interest but it is not graded for undergraduates.

Asynchronous replication option (10 points). Add a second backup-forwarding mode where the primary replies to the client BEFORE waiting for the backup's ack and forwards the write asynchronously. Report in `Report.md` how much this changes the SGD throughput number from Section 2, and discuss what guarantees you lose by doing this.

Extended measurement report (5 points). All students report the one SGD-steps-per-second number from Section 2. CS 545 students additionally report that number across the following workloads and put the table in `Report.md`:

- Varying client concurrency: 1, 4, 16 concurrent clerks running SGD against the same `"model"` key.
- Varying model dimension: 10, 100, 1000.
- Varying training loop length: 1k, 10k, 100k steps.

One short paragraph per workload explaining what you see and why.

9. Submission

Zip the whole `p2/` directory (skip any compiled binaries and any `.git/`) and upload to Canvas.

Include a `Report.md` in `p2/` with:

- Your name and NetID. If you worked with a partner, list them too.
- Your SGD-steps-per-second number from Section 2, one short paragraph.
- (CS 545 only) The extended measurement table from Section 8.
- Any AI tools you used and how; see Section 12.
- Anything else you want us to know before we grade.

`Report.md` is required for every student starting with P2. Half a page is plenty.

Do not submit files from outside `p2/`.

10. Suggested Steps

1. Re-run your P1 tests, especially the at-most-once ones from P1 Section 3.3. If any fail, fix them before you touch P2 code. See Section 1 for why this matters.
2. Read `project2-desc.pdf` (this document) from top to bottom. If primary-backup replication is a new concept, read `p2-background.pdf` next.
3. Skim `sealed.go` and `yours_test.go` so you know what you have before you write anything.
4. Run `go build .` on the skeleton. If it fails, fix your Go setup before you write any code.
5. Implement the single-server `KVServer` and `Clerk` (Part 1). Add a smoke test in `yours_test.go` that calls `Put` then `Get` and checks the value.
6. Add duplicate-detection using xids. Verify that a retried `Put` does not apply twice. Part 1 is now done.
7. Add the backup server and the `ForwardOp` RPC (Part 2). Make the primary wait for the backup's ack before replying. Test with two servers up and no failures.
8. Run `go test -race .` several times against a two-server configuration. A test that flakes one time in ten is a race you have not found yet.

After step 7: run `go test -run Perf .` on iLab once your correctness tests pass. Record the sgd*steps*per_sec number in `Report.md` (Section 2). If the test fails with "loss did not converge", your primary-backup pair is dropping updates somewhere.

CS 545 students: then add the asynchronous replication option and the extended measurement table from Section 8.

11. Tips and Resources

Read failing tests, not the diff. The test name usually tells you which invariant you broke.

Print the view number and the primary/backup identity at the top of every operation while you are debugging the view-server and failover logic in Part 2. Most bugs at this scale come from a server acting on an old view.

Save a copy of your working directory after each part passes. If you break the failover logic in Part 2 with a bad edit, you want to roll back without redoing Part 1. `git commit` is fine. `cp -r p2 p2.bak` also works.

Go documentation you will hit often:

- `net/rpc`: <https://pkg.go.dev/net/rpc>
- `sync` (WaitGroup, Mutex, Once): <https://pkg.go.dev/sync>
- `time.Ticker` for the view-server heartbeats: <https://pkg.go.dev/time>
- Race detector: <https://go.dev/doc/articles/race_detector>

The median time on this kind of project is about twelve hours. The tail is three to four times that, mostly in Part 2 failover debugging. Plan for the tail.

12. Plagiarism and AI Use

This is individual work. You may talk with others about ideas. You must write your own code. Do not share your code, during or after the course.

AI tools (Copilot, Claude, ChatGPT, or similar) are allowed only as a reference when you get stuck on Go syntax or an error message you cannot parse. You cannot paste AI-generated code into your submission.

If you use AI tools, write a short note in `Report.md`:

1. Which tools you used.
2. Which parts of the project you used them on.
3. The prompts you used, word for word.

Honest disclosures earn a small participation bonus. Not disclosing is an academic integrity violation.